



CLASS ACTIVITY-3: HAND GESTURE RECOGNITION SYSTEM

Zunaira Abdul Aziz



BSSE23058
SECTION A

1. Setup: Write a script to capture video from the webcam.

```
!pip install mediapipe==0.10.14

import cv2
import mediapipe as mp
import numpy as np
import base64
from IPython.display import display, Javascript
from google.colab.output import eval_js

# Helper to convert JS response to OpenCV Image
def js_to_image(js_reply):
    image_bytes = base64.b64decode(js_reply.split(',')[1])
    jpg_as_np = np.frombuffer(image_bytes, dtype=np.uint8)
    return cv2.imdecode(jpg_as_np, flags=1)

# JavaScript for Webcam Stream
def video_stream():
    js = Javascript('''
        var video;
        var div = null;
        var stream;
        var captureCanvas;
        var imgElement;
        var labelElement;
        var pendingResolve = null;
        var shutdown = false;

        function removeDom() {
            shutdown = true;
            if (video.srcObject) {
                video.srcObject.getTracks().forEach(track => track.stop());
            }
            if (div) div.remove();
        }

        function onAnimationFrame() {
            if (!shutdown) window.requestAnimationFrame(onAnimationFrame);
            if (pendingResolve) {
                var result = "";
                if (!shutdown) {
                    captureCanvas.getContext('2d').drawImage(video, 0, 0, 640, 480);
                    result = captureCanvas.toDataURL('image/jpeg', 0.8)
                }
                var lp = pendingResolve;
                pendingResolve = null;
                lp(result);
            }
        }
    ''')
```

```

    }

    async function createDom() {
        if (div !== null) return stream;
        div = document.createElement('div');
        document.body.appendChild(div);
        video = document.createElement('video');
        video.style.display = 'block';
        video.width = 640;
        video.height = 480;
        video.setAttribute('playsinline', '');
        video.onclick = () => { shutdown = true; };
        stream = await navigator.mediaDevices.getUserMedia({video:
{facingMode: "user"}}});
        div.appendChild(video);
        imgElement = document.createElement('img');
        imgElement.style.position = 'absolute';
        imgElement.style.zIndex = 1;
        div.appendChild(imgElement);
        video.srcObject = stream;
        await video.play();
        captureCanvas = document.createElement('canvas');
        captureCanvas.width = 640;
        captureCanvas.height = 480;
        window.requestAnimationFrame(onAnimationFrame);
        return stream;
    }

    async function stream_frame(label, imgData) {
        if (shutdown) { removeDom(); shutdown = false; return ''; }
        stream = await createDom();
        if (imgData !== "") imgElement.src = imgData;
        var result = await new Promise(resolve => { pendingResolve = resolve;
});
        return {'img': result};
    }
    ''')
    display(js)

def video_frame(label, imgData):
    return eval_js('stream_frame("{}","{}").format(label, imgData)')

```

2. Detection: Integrate MediaPipe to draw the 21 landmarks on the detected hand in real-time.

```

# Initialize MediaPipe Hands
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils

# Setup detection pipeline
hands = mp_hands.Hands(

```

```

static_image_mode=False,
max_num_hands=1,
min_detection_confidence=0.5,
min_tracking_confidence=0.5
)

```

3. Feature Extraction: Implement a function `count_fingers(landmarks)` that uses the relative positions of fingertip landmarks (e.g., landmark 8 for index finger tip) to their respective knuckles (e.g., landmark 6 for index finger PIP joint) to determine if a finger is "up" or "down". Hint: Compare the y-coordinate (for vertical gestures) or x-coordinate (for horizontal gestures).

```

def count_fingers(landmarks, hand_label):
    lm = landmarks
    index_up = lm[8].y < lm[6].y
    middle_up = lm[12].y < lm[10].y
    ring_up = lm[16].y < lm[14].y
    pinky_up = lm[20].y < lm[18].y

    thumb_up_vertically = lm[4].y < lm[3].y and lm[4].y < lm[5].y

    if hand_label == "Right":
        thumb_extended = lm[4].x < lm[3].x
    else:
        thumb_extended = lm[4].x > lm[3].x

    return index_up, middle_up, ring_up, pinky_up, thumb_up_vertically,
    thumb_extended

```

4. Rule Engine: Create an if-else logic that maps the finger count and thumb state to the gesture names.

```

def recognize_gesture(landmarks, hand_label):
    i_up, m_up, r_up, p_up, t_up_v, t_ext = count_fingers(landmarks,
    hand_label)
    fingers_up_count = sum([i_up, m_up, r_up, p_up])
    if t_up_v and fingers_up_count == 0:
        return "Thumbs Up"

    elif i_up and m_up and not r_up and not p_up:
        return "Peace"

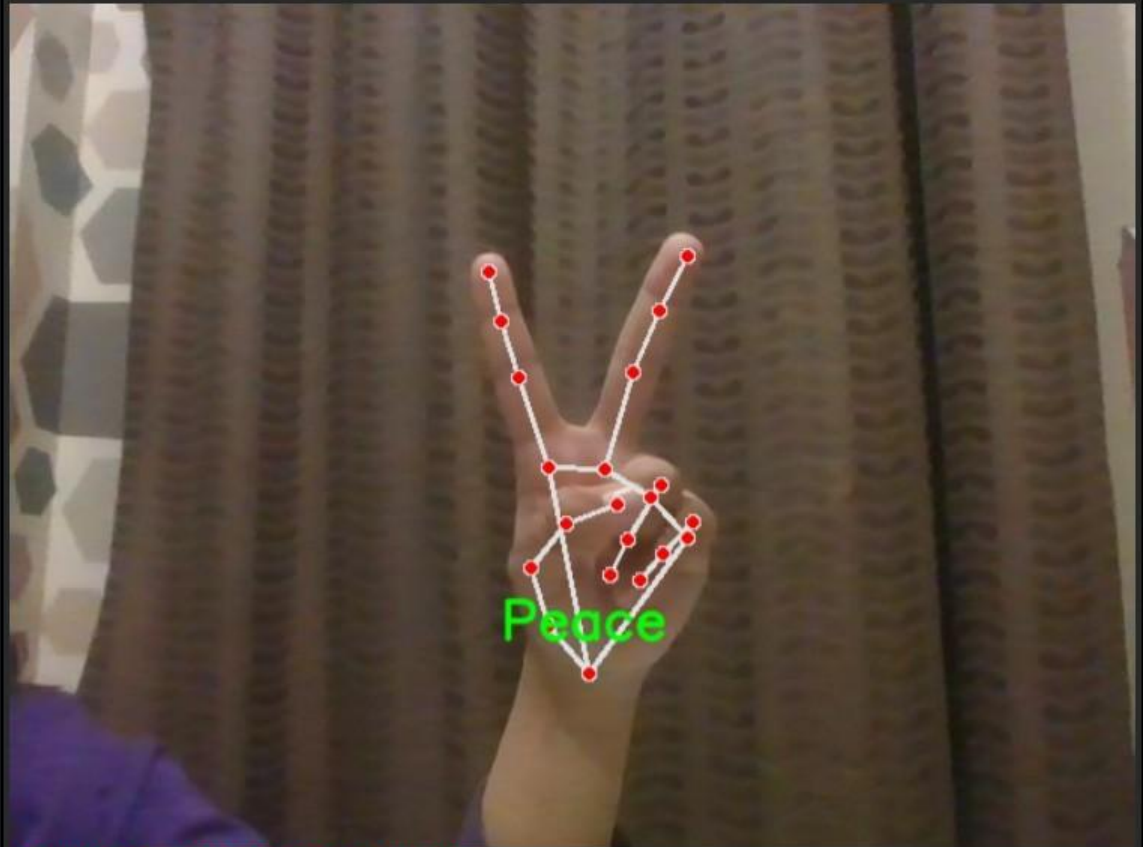
    elif t_ext and fingers_up_count == 4:
        return "Stop"

    return "Unknown"

```

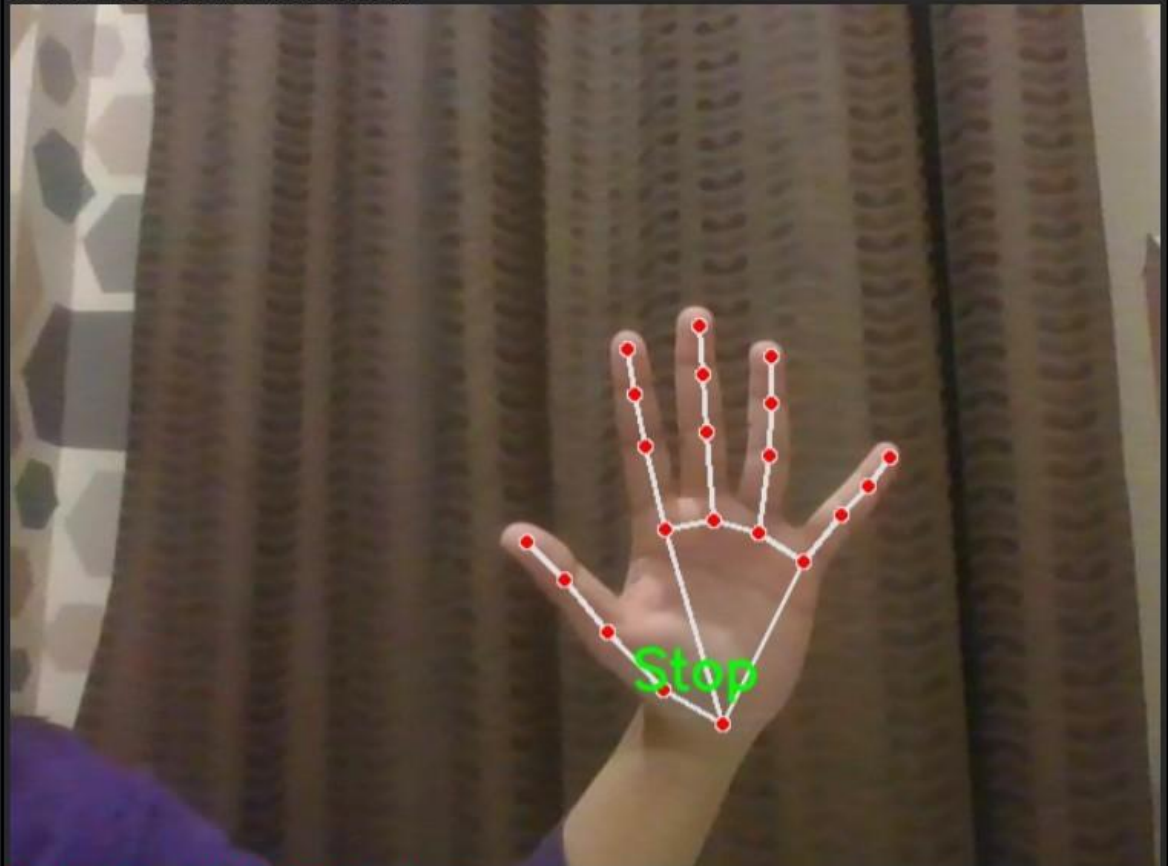
5. Display: Overlay the recognized gesture name on the video feed.

Status:Live Video Stream Active

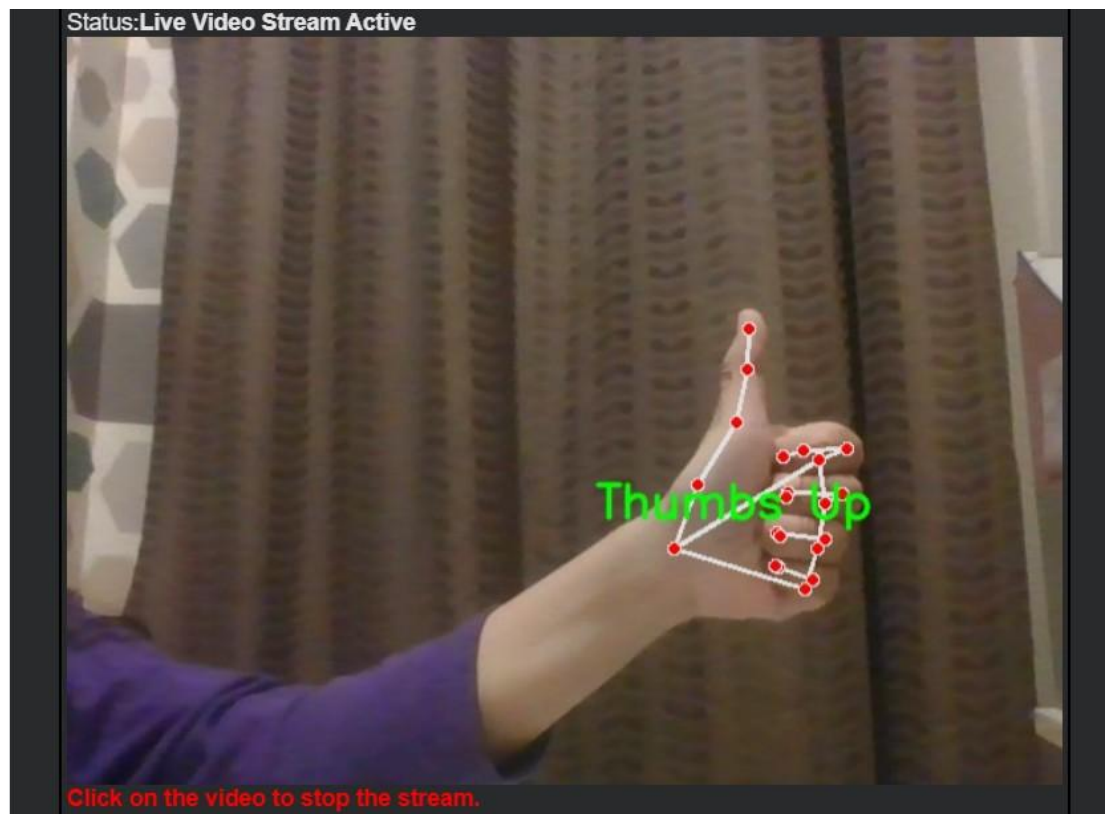


Click on the video to stop the stream.

Status:Live Video Stream Active



Click on the video to stop the stream.



WHOLE CODE

```
# -*- coding: utf-8 -*-
"""Hand Gestures.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1QuaM1NnobBXuulihfaCutXo9oU4X44P8
"""

!pip install mediapipe==0.10.14
import cv2
import mediapipe as mp
import numpy as np
import base64
from IPython.display import display, Javascript, Image
from google.colab.output import eval_js

def js_to_image(js_reply):
    image_bytes = base64.b64decode(js_reply.split(',')[1])
    jpg_as_np = np.frombuffer(image_bytes, dtype=np.uint8)
    img = cv2.imdecode(jpg_as_np, flags=1)
    return img

def bbox_to_bytes(bbox_array):
    _, bbox_encode = cv2.imencode('.jpeg', bbox_array)
```

```

        bbox_bytes = base64.b64encode(bbox_encode).decode('utf-8')
        return bbox_bytes
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
hands = mp_hands.Hands(static_image_mode=False, max_num_hands=1,
min_detection_confidence=0.5, min_tracking_confidence=0.5)

def recognize_gesture(hand_landmarks, hand_label):
    lm = hand_landmarks.landmark
    index_up = lm[8].y < lm[6].y
    middle_up = lm[12].y < lm[10].y
    ring_up = lm[16].y < lm[14].y
    pinky_up = lm[20].y < lm[18].y
    thumb_up_vertically = lm[4].y < lm[3].y and lm[4].y < lm[5].y

    if hand_label == "Right":
        thumb_extended = lm[4].x < lm[3].x
    else:
        thumb_extended = lm[4].x > lm[3].x
    fingers_up_count = sum([index_up, middle_up, ring_up, pinky_up])

    if thumb_up_vertically and fingers_up_count == 0:
        return "Thumbs Up"
    elif index_up and middle_up and not ring_up and not pinky_up:
        return "Peace"
    elif thumb_extended and fingers_up_count == 4:
        return "Stop"
    else:
        return "Unknown"

def video_stream():
    js = Javascript('''
        var video;
        var div = null;
        var stream;
        var captureCanvas;
        var imgElement;
        var labelElement;

        var pendingResolve = null;
        var shutdown = false;

        function removeDom() {
            shutdown = true;
            if (video.srcObject) {
                video.srcObject.getTracks().forEach(function(track) { track.stop();
    });
        }
    ''')

```

```

    if (div) {
        div.remove();
    }
    video = null;
    div = null;
    stream = null;
    imgElement = null;
    captureCanvas = null;
    labelElement = null;
}

function onAnimationFrame() {
    if (!shutdown) {
        window.requestAnimationFrame(onAnimationFrame);
    }
    if (pendingResolve) {
        var result = "";
        if (!shutdown) {
            captureCanvas.getContext('2d').drawImage(video, 0, 0, 640, 480);
            result = captureCanvas.toDataURL('image/jpeg', 0.8)
        }
        var lp = pendingResolve;
        pendingResolve = null;
        lp(result);
    }
}

async function createDom() {
    if (div !== null) {
        return stream;
    }
    div = document.createElement('div');
    div.style.border = '2px solid black';
    div.style.padding = '3px';
    div.style.width = '100%';
    div.style.maxWidth = '600px';
    document.body.appendChild(div);

    const modelOut = document.createElement('div');
    modelOut.innerHTML = "<span>Status:</span>";
    labelElement = document.createElement('span');
    labelElement.innerText = ' Capturing...';
    labelElement.style.fontWeight = 'bold';
    modelOut.appendChild(labelElement);
    div.appendChild(modelOut);

    video = document.createElement('video');
    video.style.display = 'block';

```



```

    video.width = div.clientWidth - 6;
    video.setAttribute('playsinline', '');
    video.onclick = () => { shutdown = true; };
    stream = await navigator.mediaDevices.getUserMedia(
        {video: { facingMode: "user"}});
    div.appendChild(video);
    imgElement = document.createElement('img');
    imgElement.style.position = 'absolute';
    imgElement.style.zIndex = 1;
    imgElement.onclick = () => { shutdown = true; };
    div.appendChild(imgElement);

    const instruction = document.createElement('div');
    instruction.innerHTML = '<span style="color: red; font-weight:
bold;">Click on the video to stop the stream.</span>';
    div.appendChild(instruction);

    video.srcObject = stream;
    await video.play();
    captureCanvas = document.createElement('canvas');
    captureCanvas.width = 640;
    captureCanvas.height = 480;
    window.requestAnimationFrame(onAnimationFrame);
    return stream;
}

async function stream_frame(label, imgData) {
    if (shutdown) {
        removeDom();
        shutdown = false;
        return '';
    }
    var preCreate = Date.now();
    stream = await createDom();

    var preShow = Date.now();
    if (label != "") {
        labelElement.innerHTML = label;
    }
    if (imgData != "") {
        var videoRect = video.getClientRects()[0];
        imgElement.style.top = videoRect.top + "px";
        imgElement.style.left = videoRect.left + "px";
        imgElement.style.width = videoRect.width + "px";
        imgElement.style.height = videoRect.height + "px";
        imgElement.src = imgData;
    }
}

```

```

        var preCapture = Date.now();
        var result = await new Promise(function(resolve, reject) {
            pendingResolve = resolve;
        });
        shutdown = false;
        return {'create': preShow - preCreate,
                'show': preCapture - preShow,
                'capture': Date.now() - preCapture,
                'img': result};
    }
    ''')
display(js)

def video_frame(label, imgData):
    data = eval_js('stream_frame("{}","{}").format(label, imgData)')
    return data

video_stream()
label_html = 'Live Video Stream Active'
overlay_image = ''

while True:
    js_reply = video_frame(label_html, overlay_image)
    if not js_reply:
        break

    img = js_to_image(js_reply["img"])

    rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = hands.process(rgb_img)

    gesture_name = "Unknown"

    if results.multi_hand_landmarks and results.multi_handedness:
        for hand_landmarks, handedness in zip(results.multi_hand_landmarks,
results.multi_handedness):
            mp_drawing.draw_landmarks(img, hand_landmarks,
mp_hands.HAND_CONNECTIONS)
            hand_label = handedness.classification[0].label
            gesture_name = recognize_gesture(hand_landmarks, hand_label)
            wrist_x = int(hand_landmarks.landmark[0].x * 640)
            wrist_y = int(hand_landmarks.landmark[0].y * 480)
            cv2.putText(img, gesture_name, (wrist_x - 50, wrist_y - 20),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,
cv2.LINE_AA)
            _, img_encode = cv2.imencode('.jpeg', img)
            img_bytes = base64.b64encode(img_encode).decode('utf-8')

```

```
overlay_image = "data:image/jpeg;base64," + img_bytes
```